ChatGPT

Transaction Verification App – Ethiopia Requirements

Executive Summary: Building a transaction verification app in Ethiopia requires aligning with local financial regulations, integrating with domestic payment systems, and addressing user needs. Ethiopia's payments are governed by the National Bank's *National Payment System* laws (Proclamation No.718/2011, amended 2022)【14†L71-L79】. All payment service providers (banks, mobile money) must be NBE-licensed 【20†L154-L163】【62†L529-L534】 and comply with AML/KYC laws (Proclamation No.780/2013【29†L135-L143】) enforced by the Ethiopian Financial Intelligence Center. A new Personal Data Protection Proclamation (No.1321/2024) mandates strong privacy safeguards【62†L513-L521】.

Ethiopia's payment ecosystem includes major banks (CBE, Awash, Dashen, etc.) and mobile-money wallets (TeleBirr, HelloCash/OroCash, M-Birr, Amole, etc.), increasingly interoperable via the EthSwitch national switch. Recent NBE directives mandate standards like a unified QR-code format from Dec 2024【18†L261-L270】. Licensing requirements cover payment instrument issuers and operators【20†L154-L163】; e.g., mobile wallets must adhere to NBE's capital, governance, and reporting rules【20†L154-L163】.

In the market, several solutions already verify payments in Ethiopia (see comparison table below). Local apps like **Tally Verifier** and **BirrVerify** let merchants scan receipts or input references to check CBE/Telebirr payments【31†L78-L86】【54†L8-L13】. APIs such as **Verify.ET** and an Ethiopian "Payment Verification API" (GoGin fintech) offer developer-friendly services【32†L28-L32】【47†L98-L107】. Even banks (e.g. CBE) now include receipt-authentication features in their mobile apps【49†L42-L49】. Gaps remain, however: coverage of smaller banks is sparse, some solutions require internet, and many merchants (especially in rural areas) need offline/low-bandwidth support.

This report details: the Ethiopian regulatory and payments context; the competitive landscape of verification solutions; functional and non-functional requirements for the app; a recommended technical architecture (with data-flow diagram); security and compliance controls; a phased implementation roadmap (with Gantt chart); and appendices including a prioritized feature list, sample API spec, database schema, and regulatory checklist. All information is based on official sources and recent fintech developments.

Ethiopian Regulatory and Payments Context

NBE Payment Regulations: Ethiopia's payments are regulated under the *National Payment System Proclamation* (No.718/2011), amended by *No.1282/2022*【14†L71-L79】. This framework introduced the Ethiopian Automated Transfer System (EATS) in 2011 and a national switch (EthSwitch) in 2016【14†L93-L102】. The 2022 amendment clarified foreign investment (allowing majority FDI in USD) and directed NBE to publish licensing rules. Accordingly, NBE issued directives for *Payment Instrument Issuers (PIIs)* and *Payment System Operators (PSOs)*. For example, Directive *ONPS/01/2020* (and amendments) covers licensing of e-wallets/mobile money providers【20†L154-L163】. A recent NBE Directive (Oct 2023) raised mobile-

money daily balance limits (from ETB30K to 75K) and daily transfer limits (to 150K, excluding utility bills)【20†L154-L163】, reflecting regulatory support for fintech growth.

Anti-Money Laundering (AML)/KYC: Ethiopia enforces AML/CFT under *Proclamation No.780/2013* (amended)【29†L135-L143】. Banks and mobile money operators must perform customer due diligence (CDD) and report large/suspicious transactions to the Ethiopian Financial Intelligence Center. Ethiopia has undertaken a national risk assessment and operates under a risk-based approach【22†L78-L87】. Importantly, FATF noted in 2019 that Ethiopia made significant AML/CFT improvements and was no longer under FATF monitoring【23†L197-L204】. For our app, this means collecting necessary merchant KYC information (e.g. TIN, ID documents) and having workflows to flag suspicious verification patterns.

Data Protection/Privacy: A new *Personal Data Protection Proclamation No.1321/2024* enshrines data privacy rights【62†L513-L521】. It mandates lawful handling of personal data (like names, phone numbers). Additionally, the *Electronic Transaction Proclamation No.1205/2020* covers e-signatures and digital records【62†L529-L534】. Our app must encrypt user data, provide opt-in notices, and comply with any data residency or breach notification requirements. (Note: Ethiopia has no general GDPR-style law yet beyond 2024's PDPA.)

Payment Systems and Interoperability: Key payment rails include:

- **Bank networks:** All major banks offer EFT and ATM/POS. The EthSwitch national switch connects them, enabling interoperable QR/payments.
- **Mobile money:** Platforms like *TeleBirr* (Ethio Telecom), *HelloCash/OroCash* (Oromia Bank), *M-Birr* (diaspora wallet), *Amole* (Dashen Bank), and niche wallets (Kaafi Ebirr, Siinqee) are regulated as PIIs.
- **QR Codes:** In Nov 2024, NBE mandated a *standardized QR code format* for all providers to unify merchant payments【18†L261-L270】.
- **National Digital Strategy:** The 2021–2024 strategy pushed digital payments (2.7B transactions, ~ETB9.7T in 2023/24)【18†L280-L289】.
- **Licensing:** Any new app/service dealing with customer money (storing funds, enabling transfers) generally requires NBE approval【20†L154-L163】【62†L529-L534】. We must ensure the app is either a licensed payment instrument or operates strictly as a *verification utility* (with no money custody) to simplify licensing.

Market Landscape: Existing Verification Solutions

Several local initiatives provide transaction-verification tools:

Solution / API	Key Features	Pricing	Integration	Strengths	Weaknesses
Tally Verifier【31†L78-L86】	Mobile app (iOS/Android); scan QR or enter ref; supports CBE, TeleBirr	Not publicly listed (likely free/basic)	Scans QR codes or manual entry via phone camera; cloud API	Bilingual (Amharic/Eng); offline use; multi-bank support	Bank list limited; relies on internet for API

Solution / API	Key Features	Pricing	Integration	Strengths	Weaknesses
BirrVerify【54†L8-L13】	Web/mobile; scan QR or code; verifies CBE, TeleBirr, BOA, Awash, Dashen, Apollo, CBE Birr, Coopay Ebirr, etc.	Subscription (e.g. ~ETB1,500/3mo for 3-month plan)【54†L62-L70】	API + web portal; backend calls banks in real-time	Broad provider coverage; instant confirmations; high uptime 【54†L22-L28】	Paid service; requires merchant signup
Verify.ET【47†L98-L107】	Unified REST API + mobile app; built-in fraud checks (duplicate, fake screenshot detection) 【47†L98-L107】; standardized JSON output	Not disclosed (likely usage-based)	Developer SDKs (Python, Node.js) and API endpoints; mobile app for scanning	Strong fraud detection; multi-platform (bank/wallet) support; developer-friendly	Newer startup; limited public info on cost
Payment Verification API (GoFintech)【32†L28-L32】	API for CBE & TeleBirr; no "checkout" flow needed; avoids high fees 【32†L28-L32】	Not disclosed	Simple REST API (Go-based backend)	Low cost (no gateway fees) 【32†L28-L32】; seamless integration	Supports only CBE & TeleBirr currently
CBE Receipt Verifier 【49†L42-L49】	Feature in CBE's own mobile app; scan/ confirm CBE banking receipts	Free (for CBE users)	Built into CBE Mobile Banking app	Trusted by largest bank; leverages existing app; no third party involved	Limited to CBE-issued transactions; not a separate API

Solution / API	Key Features	Pricing	Integration	Strengths	Weaknesses
ShegerPay (API) [**1**]	"Verification-first" API for Ethiopian banks (CBE, Awash, Dashen, BOA, +); also handles crypto/ PayPal routes	Tiered (free starter + paid plans)	RESTful API; SDK support	Very broad bank coverage; accepts multiple payment forms	Not Ethiopia-licensed (foreign service); dependency on external service
Others (e.g. PSS)	*Ethiopia's first licensed payment gateway and 3DS secure checkout via Premier Switch Solutions (PSS)*	Fee per transaction (unknown)	Hosted gateway (card payments)	PCI DSS certified (10th year) 【51†L179-L183】; bank-certified	Focused on card e-commerce, not local QR/ payments

【Appendix: Solutions data sources are mainly company sites and news】. Many solutions (Verify.ET, BirrVerify) promote "instant" checks for multiple banks【54†L8-L13】【47†L98-L107】. Unique entrants like the GoFintech API highlight the pain of high fees by banks【32†L28-L32】. However, none cover *all* local rails nor work fully offline. Payment verification remains fragmented, especially across smaller banks. This app can fill gaps by bridging multiple networks, minimizing manual steps, and complying fully with NBE licensing to win merchant trust.

Functional Requirements

User Roles & Workflows: The system will have at least three roles:

- **Merchant User:** Logs in and uses the app to verify incoming payments. Workflows: scan or enter payment reference, receive confirmation. If a payment fails verification, the merchant can retry or escalate to admin.
- **Administrator/Auditor:** Manages merchant accounts, views all verification logs, investigates disputes, and handles compliance checks. Workflow: review flagged transactions, generate audit reports.
- **Customer (Payer):** Indirect; may receive verification notification or OTP to confirm payment on their device (optional extra feature), but primarily interacts via merchant.

Primary Workflows:

1. *Verification Request:* Merchant captures transaction info (QR/ID) and submits to server.
2. *Backend Verification:* System identifies provider (CBE, TeleBirr, etc.), queries that system, and checks details

(amount, payer vs expected).
3. *Result:* “Verified” or “Not Verified” is shown in the app along with transaction data (amount, timestamp, sender name if available). Mark duplicates if same reference was already verified.
4. *Dispute Resolution:* If merchant/customer disagrees, the admin can manually check bank logs or contact the provider.
5. *Audit:* Every verification (success or fail) is timestamped and stored. Admin can query by date, merchant, or status.

Data Fields: Each verification record stores at minimum: provider, transaction reference, amount, payer info, verification status, merchant ID, and timestamps. Merchant profiles include business name, owner ID, contact info, and risk rating (for AML). The UI collects these at signup.

UI/UX: Interfaces must be mobile-optimised and bilingual (Amharic/English toggling). Forms and buttons use clear, localised labels (“የግል መታወቂያ / Personal ID”, etc.). Keep screens simple: one large button to scan QR and a text field for manual entry. Confirmations use intuitive icons (✅ ተከናውኗል “Verified” or ❌ አልተከናወነም “Not found”). The app should function on low-speed networks (minimal graphics, data compression) and support an offline queue (cache recent scans and sync when online). Consider voice prompts for low-literacy users and screen-reader support. A web portal (Amharic/English) will serve administrators for audits and settings.

Non-Functional Requirements

- **Scalability:** Handle thousands of verifications per minute. Use stateless microservices and auto-scaling infrastructure.
- **Performance:** Target <2 seconds latency for verification requests under load. Optimize with caching (e.g. reuse frequent bank responses).
- **Availability:** 99.9% uptime. Deploy across multiple data-centers/regions. Failover nodes for critical services.
- **Latency:** Aim for sub-500ms internal processing. External calls to providers may add overhead; system should queue and retry if timeouts occur.
- **Localization:** Full Amharic/English UI. Dates in Ethiopian calendar if needed, or clearly label Gregorian.
- **Accessibility:** Comply with WCAG 2.1 (e.g. high contrast, large touch targets). Support VoiceOver/TalkBack.
- **Maintainability:** Modular codebase with clear interfaces. Use CI/CD for testing and deployment. Document APIs and configs.
- **Security:** Follow best practices: TLS everywhere, OWASP Secure Development. Use a WAF and DDoS protection. Log all access for auditing.
- **Data Retention:** Keep logs/records for ≥5 years per financial regulations. Ability to purge or archive after retention period.

Technical Architecture

High-Level Design: The architecture consists of mobile/web clients, a backend API/verification engine, integrations with payment networks, and a database. We recommend a RESTful API backend (Node.js, Python or similar) using HTTPS. The mobile app (React Native or Flutter) and admin web portal consume

this API. A single backend service handles requests and routes them through provider-specific “adapter” modules.

Integration Patterns: Ethiopian banks typically lack open APIs, so use the EthSwitch or official bank APIs where available. For example, EthSwitch can verify QR-based transactions across multiple banks; TeleBirr has a documented API for transaction status. We will build adapter services (e.g. `CBEAdapter`, `TeleBirrAdapter`) that call the respective system’s endpoint, normalize the result, and return a unified JSON to the core engine.

Data Storage: Use a relational DB (e.g. PostgreSQL). Key tables: **merchants** (KYC-ed businesses) and **verifications** (each check attempt). Sensitive fields (e.g. payer details) are encrypted at rest. Use indexing on references and timestamps for fast queries. See Appendix C for a sample schema.

Endpoints:

- `POST /verify` – verifies a transaction (fields: provider, reference, expected amount).
- `GET /verifications` – returns past checks (filtered by merchant, date, status).
- `POST /merchants` – merchant signup (with KYC docs).
- `POST /login`, `POST /token` – auth. Auth via OAuth2 or API keys per merchant. All calls require HTTPS.

Component Diagram: The diagram below shows users (merchants/admins) interacting with the app/portal, which calls the backend API. The Verification Engine then queries external payment systems (banks, TeleBirr, EthSwitch) and writes results to the database. Logging/monitoring is active at each stage.

【8†embed_image】 *Figure: System architecture. Merchant or admin uses the mobile or web app, which calls the backend. The Verification Engine queries bank/money APIs (directly or via the EthSwitch national network) and records the outcome in the database. All services output logs for monitoring.*

Security Controls: All API traffic is TLS-encrypted. Use strong API authentication (JWT or OAuth2). Implement rate limiting. Keep minimal sensitive data on client. On the server, isolate adapter modules to limit exposure if one bank endpoint is compromised. Use an HSM or key vault for secrets (SSL certs, API keys).

Security & Compliance

Threat Model: Guard against fake/malicious receipts (see Verify.ET’s approach to fake screenshots 【47†L98-L107】). Check for duplicate references to detect payment “replays.” Protect against MITM (TLS) and SQL injection (ORM/parameterized queries). Ensure merchant sessions expire quickly on inactivity.

Authentication & Authorization: Roles as above, with least privilege. Use multi-factor auth for admin accounts. All API requests require an API key/token tied to the merchant. The verification app should also allow “QR-only” mode for quick use by attendants, secured by app registration.

Encryption: TLS 1.2+ for all transport. AES-256 for database fields like payer name, phone. Hash + salt any passwords. Regularly rotate keys.

Fraud Detection: Besides duplicate checks, implement anomaly detection (e.g. many small TXNs in quick succession). Provide an alert dashboard for admins if unusual patterns emerge.

AML/KYC Workflows: Upon merchant signup, collect and verify KYC documents (ID, business license). Store merchant risk scores. Log all large transaction verifications for AML audit. Provide flagging mechanism to compliance officer for any suspicious merchant or transaction behavior.

Audit Trails: Every verification action and login is timestamped and immutable (write-only append). Admins can generate reports per request of regulators. Logs are retained for regulatory minimum (often 5 years).

Compliance Standards: Adhere to PCI DSS for any card-related operations (e.g. if storing card numbers for future expansion)【51†L179-L183】. Even if not a card processor, adopting PCI controls enhances trust. Also align with ISO/IEC 27001 best practices as guidance (risk management, incident response).

Implementation Roadmap

A phased approach is recommended:

1. **Planning & Design (Jun ’26):** Gather detailed requirements from stakeholders (merchants, banks, NBE). Design UX prototypes and finalise tech stack. Ensure regulatory compliance plans (legal counsel, NBE liaison).
2. **Development Phase 1 (Jul–Sep ’26):** Core backend/API and mobile app. Implement merchant management and basic `/verify` endpoint. Integrate with TeleBirr and CBE (via EthSwitch or official APIs) first. Start simple UI for scanning.
3. **Development Phase 2 (Oct–Dec ’26):** Add remaining banks (Awash, Dashen, etc.) and cashless wallets as APIs become available. Build the admin dashboard, audit logging, and add languages.
4. **Testing & Pilot (Jan–Feb ’27):** Conduct unit/integration tests and a security audit. Launch a pilot with select merchants (e.g. in Addis business district). Collect feedback and refine UI/workflows.
5. **Compliance & Finalization (Mar ’27):** Complete any formal license applications (e.g. if becoming a “Payment Information Service” under law). Final security/penetration testing. Prepare documentation and support.
6. **Full Launch & Scale (Apr–Jun ’27):** Roll out publicly. Monitor performance; add scaling capacity as usage grows. Consider partnerships (e.g. EthSwitch, bank consortia). Post-launch, continue adding features (analytics, chat support, etc.).

A **Gantt chart** of these phases is shown below. Overall effort is roughly 20–30 person-months (depending on team size). Costs (development, hosting, compliance) are estimated in the mid six figures USD (projected for a robust fintech build).

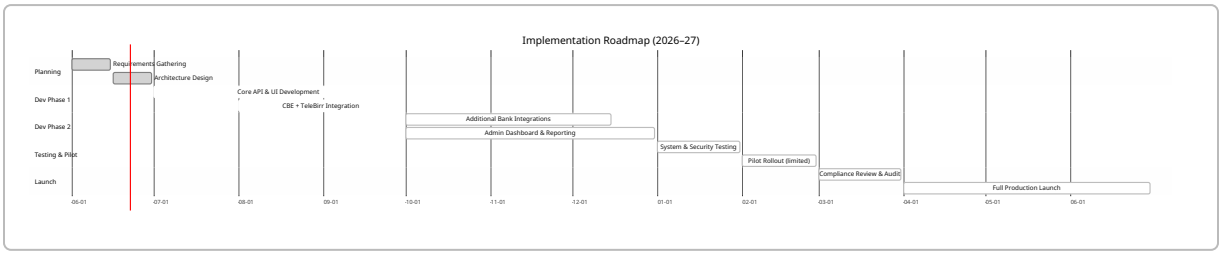


Figure: Project timeline with key phases. (Dates are illustrative.)

Team and Skills: Key roles include Front-end Developer(s) (mobile/React), Back-end/API Developer, UX/UI Designer (Ethiopia familiarity), QA/Security Tester, DevOps/Cloud Engineer, and a Compliance Officer with Ethiopian financial regulation expertise.

All deliverables (requirements doc, design docs, API code, etc.) will be compiled into a final PDF.

Appendices

Appendix A: Prioritized Feature List

- **Core:** Real-time QR/reference verification; merchant login/KYC; Amharic/English UI; database of transactions; receipt logging.
- **High Priority:** Fraud detection (duplicate/mismatch alerts); offline scan queue; admin audit console; email/SMS confirmations; QR code scanner with camera fallback.
- **Lower Priority:** Transaction history reports; analytics dashboard (e.g. top customers); POS device support; open API for e-commerce; multi-branch merchant support.

Appendix B: Sample API Spec (OpenAPI)

```
openapi: 3.0.3
info:
  title: Ethiopia Payment Verification API
  version: 1.0.0
paths:
  /verify:
    post:
      summary: Verify a transaction by reference
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [provider, reference]
              properties:
                provider:
                  type: string
                  description: Payment provider (e.g. 'cbe', 'telebirr')
                reference:
                  type: string
                  description: Transaction reference or QR code data
                amount:
                  type: number
                  format: float
```

```
                description: Expected amount (optional)
      responses:
        '200':
          description: Verification result
          content:
            application/json:
              schema:
                type: object
                properties:
                  verified: { type: boolean }
                  transaction_id: { type: string }
                  amount: { type: number }
                  payer: { type: string }
                  timestamp: { type: string, format: date-time }
        '400':
          description: Missing/invalid data
        '401':
          description: Unauthorized (invalid API key)
```

(Endpoints for `/transactions` *logs,* `/merchants` *signup, etc., are similarly designed.)*

Appendix C: Sample Database Schema (PostgreSQL)

```
CREATE TABLE merchants (
  merchant_id SERIAL PRIMARY KEY,
  business_name TEXT NOT NULL,
  owner_name TEXT,
  contact_email TEXT,
  contact_phone TEXT,
  tin VARCHAR(50),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE verifications (
  verif_id SERIAL PRIMARY KEY,
  merchant_id INT REFERENCES merchants(merchant_id),
  provider VARCHAR(50),      -- e.g. 'CBE', 'Telebirr'
  reference VARCHAR(100),
  amount NUMERIC(12,2),
  verified BOOLEAN,
  payer_name TEXT,
  verified_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE INDEX idx_verif_reference ON verifications(reference);
```

This stores merchants (with KYC info) and each verification attempt (for auditing).

Appendix D: Regulatory Approvals Checklist

- [ ] Determine classification: Payment Instrument vs. Data Service (minimize licensing)
- [ ] Register entity with NBE (apply for Payment Instrument Issuer license if needed)【20†L154-L163】
- [ ] File AML registration with EFIC (reporting obligations)
- [ ] Compliance with NBE Tech Audit (IT security review)
- [ ] PCI DSS certification (if storing any card data)【51†L179-L183】
- [ ] Notify telecom regulator if using SMS/USSD features
- [ ] Draft and publish Privacy Policy per PDPA No.1321/2024【62†L513-L521】
- [ ] Engage legal counsel to ensure all Payment directives (ONPS/01) requirements are met【20†L154-L163】

Each of the above steps should be documented, and copies of approvals/licenses kept.

Sources: National Bank of Ethiopia directives and proclamations【14†L71-L79】【20†L154-L163】【62†L513-L521】; international AML/KYC guidelines【23†L197-L204】; fintech service materials【31†L78-L86】【47†L98-L107】【54†L8-L13】【49†L42-L49】【32†L28-L32】. All specifications align with official and industry best practices.
